

Informe de arquitectura y alcance

Software Ganadero

Documentación técnica para asesoría externa de arquitectura y alcance

Rama analizada: `claude/eager-gauss-0dBpk`

Método: lectura directa del código · compilación y ejecución real de la suite de pruebas

Cada afirmación cita el archivo que la sustenta. Lo no confirmable se marca «NO VERIFICADO».

Informe de arquitectura y alcance — Software Ganadero

Verificado leyendo el código de la rama `claude/eager-gauss-0dBPk`. Cada afirmación cita el archivo que la sustenta. Lo no confirmable se marca **NO VERIFICADO**.

1. Estructura y compilación

Árbol (3 niveles, sin `build/`, `vcpkg_installed/`, `src/third_party/`):

```
src/
├─ application/    (dto/ · session/ · usecases/{usuario,finca,ganado,produccion})
├─ domain/        (entities/ · repositories/)
├─ infrastructure/(database/ · repositories/ · utils/)
├─ presentation/  (viewmodels/)
├─ ui/            (main.qml · screens/)
└─ tests/         (repositories/ · usecases/{usuario,finca,ganado,produccion})
```

Compilación — `CMakeLists.txt`, `CMakePresets.json`:

- Estándar **C++20** (`CMakeLists.txt:4`, `CMakePresets.json:11`).
- Compilador **MinGW g++ 13.1.0** (`CMakePresets.json:13`), **Qt 6.11.1** (`CMakePresets.json:12`).
- Dependencias vía `vcpkg`: `sqlite3`, `lmbd`, `nlohmann-json` (`vcpkg.json`); `GoogleTest` vía `FetchContent` con gate `BUILD_TESTING`.
- Flag propio `NO_CACHEGEN` en `qt_add_qml_module` (`src/CMakeLists.txt:39`) para evitar OOM al compilar QML grandes con MinGW.

¿Compila hoy?

- Las 12 fuentes no-Qt (repositorios, casos de uso, utils), `LMDB` (`src/third_party/lmdb/mdb.c`) y `SQLite` del sistema **compilan y enlazan sin errores** — verificado en este análisis; los 8 binarios de test enlazaron y la suite corre (ver §9).
- El binario Qt completo **no se pudo compilar en este entorno** (sin Qt6): `AppViewModel.cpp` requiere `<QObject>`. **NO VERIFICADO aquí**, pero durante esta sesión el `.exe` sí se generó en la máquina Windows del equipo (MinGW+Qt), con un único ajuste de runtime (renombrar `liblmbd.dll` → `lmbd.dll`).

2. Modelo de datos SQLite

DDL real en `src/infrastructure/database/SQLiteDatabase.cpp`, método `initSchema()` (líneas 29-76).
3 tablas, creadas con `CREATE TABLE IF NOT EXISTS`:

Tabla	Columnas (tipo)	PK / FK
usuarios	id TEXT, nombre TEXT NN, email TEXT NN, contrasena TEXT NN, fecha_registro TEXT, fecha_ultimo_backup TEXT	PK id
fincas	id TEXT, nombre TEXT NN, n_hectareas INT NN, n_potreros INT NN, capacidad INT NN, id_usuario TEXT NN	PK id · FK id_usuario → usuarios(id)

ganado	id TEXT, especie TEXT NN, identificador TEXT NN, id_usuario TEXT NN, id_finca TEXT NN, nacimiento TEXT NN, sexo TEXT NN, estado TEXT NN, raza TEXT, id_padre TEXT, id_madre TEXT, chapeta TEXT, fecha_destete TEXT, foto BLOB, fecha_ultimo_parto TEXT, fecha_ultima_palpacion TEXT, fecha_inseminacion TEXT	PK id · FK id_usuario → usuarios(id), id_finca → fincas(id) · id_padre/id_madre → ganado(id)
--------	--	---

- **Índices:** ninguno explícito (`grep "CREATE INDEX"` → vacío). Solo los implícitos de PRIMARY KEY.
- **UNIQUE:** ninguno. `email` **no** tiene unicidad a nivel de BD (se valida en app, ver §5).
- **PRAGMA** en el constructor (`SQLiteDatabase.cpp:12-13`): `journal_mode=WAL` y `foreign_keys=ON` .
- **Control de versiones del esquema:** **no existe**. Sin tabla de migraciones ni `PRAGMA user_version` . El esquema se crea idempotentemente al arrancar.

3. Modelo de datos LMDB

Verificado en `src/infrastructure/repositories/LmdbProduccionRepository.cpp` y `LmdbDatabase.cpp` .

- **Qué se guarda:** la producción de cada animal (preñez, ordeño, lista de partos, registros de leche, registros de carne).
- **Formato de clave:** el **UUID del animal, crudo, sin prefijo** — `MDB_val key{ p.id.size(), p.id.data() }` (`save :91, load :67`). Mismo `id` que la fila en `ganado` (`Produccion.h:18`).
- **Serialización del valor:** JSON con `nlohmann ( toJson , :16-35)`. Claves: `id`, `prenez`, `ordeno`, `partos[] {id, fecha}`, `registro_leche[] {fecha, valor}`, `registro_carne[] {fecha, valor}` .
- **Unidad de escritura — DETERMINANTE:** el **historial completo del animal bajo una sola clave**. No hay una entrada por evento. Cada operación es *read-modify-write* del documento entero (`addRegistroLeche :172-178`; `save` reescribe todo con `mdb_put` , `:80-98`). Insertar el registro j-ésimo re-serializa j registros → $O(M)$ por animal.
- **Índices secundarios:** ninguno. `mdb_dbi_open(txn, nullptr, 0, &m_dbi)` (`LmdbDatabase.cpp:22`) abre la DBI por defecto sin flags. Toda consulta pasa por la clave = UUID.
- **Tamaño del mapa:** fijo 256 MB por defecto (`LmdbDatabase.h:10`).

4. Capas y contratos

Casos de uso por módulo (headers en `src/application/usecases/`):

Módulo	Casos de uso	Firma (ejemplo)
Usuario	Create, Login, GetAll, GetById, Update, Delete, CheckEmailExists	Login: <code>LoginDto</code> → <code>optional<UsuarioResultDto></code> (<code>UsuarioUseCases.cpp:57</code>)
Finca	Create, GetAll, GetById, Update, Delete	Create: <code>CreateFincaDto</code> → <code>optional<FincaResultDto></code>
Ganado	Create, GetAll, GetById, Update, Delete, GetByFinca, ValidarProgenitores, ActualizarFechaPartaMadre	Create: <code>CreateGanadoDto</code> → <code>optional<GanadoResultDto></code> (<code>GanadoUseCases.h:14</code>)
Producción	GetProduccion, Update{Prenez,Ordeno}, {Add,Update,Delete}Parto, {Add,Update,Delete}RegistroLeche, ...Carne	AddRegistroLeche: <code>(id, RegistroFechaDto)</code> → <code>bool</code> (<code>ProduccionUseCases.h:69-74</code>)

Manejo de errores en los contratos: sin excepciones de negocio ni tipos de error. Éxito/fallo con `std::optional<T>` (consultas/creaciones) o `bool` (mutaciones). Sin códigos ni mensajes a nivel de caso de uso.

Interfaces → implementación (`src/domain/repositories/` → `src/infrastructure/repositories/`):

- `IUsuarioRepository` → `SqliteUsuarioRepository` (`getAll`, `getById`, `getEmail`, `insert`, `update`, `deleteById`).
- `IFincaRepository` → `SqliteFincaRepository` (`getAll`, `getById`, `insert`, `update`, `deleteById`).
- `IGanadoRepository` → `SqliteGanadoRepository` (+ `getByFinca`).
- `IProduccionRepository` → `LmdbProduccionRepository` (`getById`, `insert`, `deleteById`, `update{Prenez,Ordeno}, ...Parto, ...RegistroLeche/Carne`).

Independencia del dominio: verificada. Todos los `#include` de `src/domain/` son solo STL y cabeceras propias. `grep "sqlite3|lmdb|nlohmann|json|Q_OBJECT|QString"` sobre `src/domain/` → **vacío**. No hay violaciones: ni SQL, ni JSON, ni Qt filtrados al dominio. La conversión `enum ↔ string` vive en el dominio (`GanadoEnums.h`), lo cual es lógica de dominio pura.

5. Puente C++ ↔ QML

- **Clase expuesta:** una sola, `Presentation::AppViewModel : QObject` (`AppViewModel.h:18`).
- **Registro:** por contexto, no por tipo. `engine.rootContext()->setContextProperty("appViewModel", appViewModel.get())` (`main.cpp:118`). No usa `qmlRegisterType`.
- **Propiedades a QML** (`Q_PROPERTY`, :22-27): `currentScreen` (NOTIFY `currentScreenChanged`), `isLoggedIn` y `userName` (NOTIFY `sessionChanged`).
- **Métodos invocables:** ~40 `Q_INVOKABLE` con tipos Qt — ej. `createGanado(QVariantMap)`, `getProduccion(QString)→QVariantMap`, `addRegistroLeche(QString,QString,double)`.
- **Propagación a la UI:** señales `currentScreenChanged` y `sessionChanged`. La navegación se resuelve en `main.qml:26-38` (`onCurrentScreenChanged` → `stackView.replace(...)`). Las listas se refrescan **manualmente** (los `.qml` llaman `recargar()`), no hay modelos reactivos.
- **Errores al usuario:** señal `errorOccurred(QString)` (`AppViewModel.h:154`), emitida en 15 puntos (ej. :115, :128, :140). Las vistas la escuchan con `Connections{ onErrorOccurred }` y muestran un banner rojo.

6. Interfaz

14 archivos QML (CMake declara 14; hay 14 reales). Enrutados por `StackView` en `main.qml`.

Pantalla	Qué hace	Se conecta a
<code>main.qml</code>	Ventana raíz, <code>StackView</code> , enrutado por <code>currentScreen</code>	<code>AppViewModel</code> (props/señales)
<code>LoginScreen</code>	Inicio de sesión	<code>login</code> , <code>goToRegister</code>
<code>RegisterScreen</code>	Crear cuenta	<code>createAccount</code>
<code>MenuScreen</code>	Menú principal (hato + análisis)	<code>goToInventario/Reproductivo/Leche/Peso</code> , <code>logout</code>
<code>InventarioScreen</code>	CRUD de fincas (grid + modal)	<code>getFincas</code> , <code>createFinca</code> , <code>updateFinca</code> , <code>deleteFinca</code>

RegistroReproductivoScreen	Lista de animales → detalle	getAllGanado; abre DetalleAnimalComponent
RegistroLecheScreen	Lista de animales → registro leche	getAllGanado; abre RegistroLecheAnimalView
RegistroPesoScreen	Lista de animales → registro carne/peso	getAllGanado; abre RegistroCarneAnimalView
DetalleAnimalComponent	Ficha: campos, foto, preñez/ordeño	getGanado, getProduccion, updatePrenez, updateOrdeno, updateFoto
NuevoAnimalModal	Alta/edición de animal (714 líneas)	createGanado, getRazasPorEspecie, validarProgenitores
EditarCampoModal	Edición de un campo individual	updateGanado, getEspecies/Sexos/Estados
RegistroLecheAnimalView	Tabla add/update/delete de leche	getProduccion, addRegistroLeche, updateRegistroLeche, deleteRegistroLeche
RegistroCarneAnimalView	Tabla add/update/delete de carne/peso	getProduccion, addRegistroCarne, updateRegistroCarne, deleteRegistroCarne
CalendarPopup	Selector de fecha reutilizable	— (componente UI puro)

Deshabilitadas / solo maqueta (`MenuScreen.qml:125-127`): botones «**Herramienta visual**», «**Alertas**», «**Proyecciones**» con `enabled: false` y sin pantalla ni backend. `Qt6::Charts` se enlaza en CMake pero **ningún QML lo importa** (`grep "import QtCharts" → vacío`): no hay gráficos.

7. Estado real por módulo

Módulo	Dominio	Caso de uso	Repositorio	UI	Pruebas
Usuario / login	completo	completo	completo	parcial ¹	18+10
Finca	completo	completo	completo	completo	9+6
Ganado (inventario)	completo	completo	completo	completo	26+11
Genealogía (padre/madre)	completo	completo	completo	parcial ²	parcial
Producción – leche	completo	completo	completo	completo	sí
Producción – carne/peso	completo	completo	completo	completo	sí
Reproductivo – preñez/ordeño	completo	completo	completo	completo	16+13
Reproductivo – partos	completo	completo	completo	inexistente ³	sí
Alertas / gráficos / proyecciones	—	—	—	maqueta	—
Sincronización / backup	inexistente ⁴	—	—	—	—

¹ Sin pantalla de editar perfil ni cambiar contraseña (`updateAccount` existe en backend, sin UI). ² `validarProgenitores` se usa en `NuevoAnimalModal`, pero `getGanadoByFinca` no se usa en UI (se usa `getAllGanado`). ³ `addParto` / `updateParto` / `deleteParto` existen y están probados, pero ningún QML los llama; la UI solo maneja el campo suelto `fechaUltimoParto`. ⁴ Solo la columna `fecha_ultimo_backup`, sin lógica.

Lo que un usuario NO puede hacer hoy, aunque exista lógica de respaldo:

- Registrar/editar/eliminar **partos** de un animal (backend + pruebas listos, sin pantalla).
- **Editar su cuenta o cambiar contraseña** (`updateAccount` sin UI).

- Ver cualquier **gráfico, alerta o proyección** (maquetas deshabilitadas).
- **Respaldar/sincronizar** datos (sin implementación).
- Filtrar ganado por finca desde una consulta indexada (`getGanadoByFinca` sin UI; la app trae todo con `getAllGanado`).

8. Viabilidad de tres adiciones

a) Importador Excel/CSV

- **Lectura de archivos existente:** solo `AppViewModel::leerArchivoBase64` (`AppViewModel.cpp:216`), que abre un `QFile` **para convertir imágenes a base64**, no datos tabulares. **No hay parser CSV ni Excel.** `ImageProcessor` solo redimensiona imágenes (`ImageProcessor.h:12`).
- **Librería a agregar:** CSV no requiere librería (`QTextStream` /parseo manual). Excel `.xlsx` sí requiere nueva dependencia (p. ej. `QXlsx`) vía `vcpkg` — no presente hoy.
- **Punto de entrada (sin saltarse validaciones):** un nuevo caso de uso `application/usecases/import/ImportUseCase` que, por fila, llame a los casos de uso existentes (`CreateGanadoUseCase` , `CreateFincaUseCase` , `AddRegistroLecheUseCase` , `AddRegistroCarneUseCase`). Nunca escribir directo a los repositorios.
- **Archivos a tocar/crear:** crear `ImportUseCase` (+interfaz); `Q_INVOKABLE` `importarArchivo(QString)` en `AppViewModel` ; pantalla/botón QML con `FileDialog` (ya se importa `Qt.labs.platform` en `NuevoAnimalModal.qml:4`).
- **Riesgos:** formatos de fecha inconsistentes (leche/carne `aaaa-mm-dd` , partos `dd/mm/aaaa`); validar `especie / raza` contra enums; FK exigen finca/usuario previos; sin transacción de importación (fallo a mitad deja datos parciales).
- **Estimación:** CSV **8-12 h**; añadir Excel **+4-6 h**.

b) Módulo de alertas sobre datos existentes

- **Consultas que soportan hoy los repositorios:** ganado → `getAll` , `getById` , `getByFinca` ; producción → **solo `getById(uuid)`** (`IProduccionRepository.h`). No hay consultas por rango de fechas ni agregaciones.
- **Calculable sin entidades nuevas:** «animal sin pesaje en N días» (recorrer `getAllGanado` + `getProduccion` por animal, fecha máx. de `registroCarne` — $O(N)$ lecturas, sin índice); «caída de leche vs su promedio» (promediar `registroLeche` y comparar); «días desde último parto» (`fechaUltimoParto` o lista `partos`).
- **Consultas a agregar:** ninguna imprescindible (se computa en memoria); para eficiencia convendría un método que devuelva todas las producciones o un índice por fecha (hoy no existe).
- **Archivos a tocar/crear:** `application/usecases/alertas/AlertasUseCase` (cálculo puro sobre repos existentes); `Q_INVOKABLE` `getAlertas()`→`QVariantList` ; pantalla QML (el botón «Alertas» ya existe deshabilitado, `MenuScreen.qml:126`). Sin entidad de dominio nueva.
- **Riesgos:** rendimiento (carga el JSON completo de cada animal), dos formatos de fecha, definir umbrales configurables.
- **Estimación:** **10-16 h**.

c) Iconografía en la interfaz

- **Uso actual:** emoji/símbolos Unicode como texto de botón, verificados en los .qml: 🗑️ (U+1F4C5), 🗑️ (U+1F5D1), 🗑️ (U+270E), ✖️ (U+2715), ⬅️ (U+2190), ➡️ (U+2192). No hay archivos de icono (SVG/PNG) en el repo.
- **Carga de imágenes:** las fotos se cargan como **data URI base64**, no desde archivos empaquetados:

```
Image { source: "data:image/png;base64," + appViewModel.getFotoBase64(id) }
```


(DetalleAnimalComponent.qml:136-145). `resources.qrc` **solo empaqueta ui/main.qml**, ningún asset de icono.
- **¿QML renderiza emoji correctamente en el equipo de desarrollo? NO VERIFICADO** desde este entorno (no puedo ejecutar la GUI aquí). Lo confirmable: los glifos están en el código y la app se ejecutó en la máquina Windows del equipo durante esta sesión. El render de emoji a color depende de la fuente del SO (en Windows, Segoe UI Emoji) y en Qt ha sido históricamente irregular — **debe confirmarse en la máquina objetivo** antes de depender de ellos.
- **Para iconografía robusta:** añadir SVG/PNG a `resources.qrc` y referenciarlos con `source: "qrc:/icons/..."`, sustituyendo los emoji de texto.
- **Estimación: 4-8 h** según el set de iconos.

9. Deuda técnica y riesgos (por gravedad)

1. **Contraseñas en texto plano.** `UsuarioUseCases.cpp:35` (copia literal al crear) y `:61` (comparación en claro al login); persistida con `bind_text ( SQLiteUsuarioRepository.cpp:84 )`. **Riesgo:** leer `ganadero.db` expone todas las credenciales; sin hashing ni sal.
2. **Sin integridad entre los dos motores.** SQLite (ganado) y LMDB (producción) no comparten transacción. Al crear, el insert de producción **no se comprueba**: `m_produccionRepo->insert(p);` (`GanadoUseCases.cpp:119`) — si falla, queda un animal sin producción y el caso de uso retorna éxito. Al borrar (`:201-202`) se borra producción y luego ganado sin atomicidad ni rollback. **Riesgo:** documentos huérfanos o faltantes, indetectables.
3. **Escritura O(M) en LMDB (read-modify-write).** Cada inserción reescribe el documento completo del animal (`LmdbProduccionRepository.cpp:172-178 + save :80-98`). Además `save` imprime por `std::cerr` en cada escritura (`:81-84`). **Riesgo:** degradación acumulada con historiales largos (medido: 10.000 registros $\approx$ 15,5 s) y ruido/coste de I/O por los logs.
4. **Login ineficiente y sin unicidad en BD.** `LoginUseCase` hace `getAll()` y recorre linealmente comparando en claro (`UsuarioUseCases.cpp:59-63`); `email` sin índice ni `UNIQUE`. La unicidad se valida solo en app (`CheckEmailExists`, `AppViewModel.cpp:128`). **Riesgo:** $O(n)$ por login y emails duplicados si se inserta por fuera del flujo de la app.
5. **Manejo de errores pobre y pruebas parciales.** Los casos de uso devuelven `bool / optional` sin causa; `std::cerr` de depuración disperso (`UsuarioUseCases.cpp:40-42`, `LmdbProduccionRepository.cpp:81-84`); `mapSize` de LMDB fijo (256 MB) con `MDB_MAP_FULL` no manejado. Las **109 pruebas son de integración** (repos + casos de uso contra BD reales efímeras); **no hay pruebas de UI ni del AppViewModel**.

Lo más frágil hoy: la partición en dos motores sin escritura atómica — un animal puede existir en SQLite sin su producción en LMDB (o al revés) y nada lo detecta ni lo repara. Sumado a las contraseñas en texto

plano y al reescribir el documento completo en cada registro ($O(M)$), la capa de datos es el punto débil del sistema. La lógica de negocio, en cambio, está bien separada y probada.